Auckland University of Technology (AUT)

Data Engineering and AI - ENGE707

Kaggle Competition: Facial Feature Classification Challenge Report

By: Aman Mohammed – 24246816

Table of Contents

Table of Contents....2
Introduction....3
Dataset Exploration and Understanding....4
Data Quality Checks....5
Training and Validation Split....6
Data Preprocessing and Pipeline Preparation....7
Model Selection and Architecture....8
Architecture Details....8
Training Strategy and Overfitting Management....9
Training Approach....9
Hyperparameter Justification....9
Training Behavior and Generalization....10
Model Evaluation and Kaggle Submission....11
Validation Performance....11
Confusion Matrix Analysis....11
Kaggle Submission Result....12
Results Analysis and Improvement Strategies....14
Strengths and Limitations....14
Possible Improvements....14
Conclusion....15
References....16
Appendix....17
Appendix A – GitHub Repository....17
https://github.com/Asura-002/ENGE707-Facial-Feature-Classification.git....17
Appendix B – Kaggle Username....17

Introduction

Image classification is a computer vision task in which a model learns to assign an image to one of several predefined categories. In this project, the aim was to develop a deep learning pipeline capable of classifying cropped facial features from cats and dogs. The dataset contained eight target classes: CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth and DogNose. The final model also needed to generate predictions for 29,443 unseen test images in the required Kaggle submission format.

The project followed a complete machine learning workflow, beginning with dataset exploration and quality checks. The images were then preprocessed, divided into training and validation sets, and loaded through a reproducible data pipeline. An ImageNet-pretrained EfficientNet-B0 model was selected because it provides a useful balance between classification performance and computational cost. The model was first trained with its feature-extraction layers frozen and was then fine-tuned using a smaller learning rate.

The final model achieved a validation accuracy of 90.17% and a balanced validation accuracy of 90.01%. Its Kaggle public leaderboard accuracy was 88.24%. This report describes the dataset, preprocessing decisions, model architecture, training strategy, and evaluation results. It also analyses the model's common errors and discusses realistic improvements that could be made in future work.

Dataset Exploration and Understanding

The provided dataset contained 63,660 labelled training images and 29,443 unlabeled test images. Each training image was linked to one of eight target classes through train_labels.csv, while sample_submission.csv defined the required structure and image order for the final Kaggle submission. Initial checks confirmed that the number of training files matched the number of labels and that every image identifier was unique and correctly linked to a file.

The class distribution was not completely balanced. CatEye was the largest class with 13,884 images, followed by DogEye with 11,580 images. CatFace, CatMouth and CatNose each contained 6,942 images, while DogFace, DogMouth and DogNose each contained 5,790 images. Therefore, the largest class contained approximately 2.4 times as many images as the smallest classes. Although every class had several thousand examples, this difference could cause the model to favor the larger eye classes if it was not addressed during training.

Representative images showed considerable variation in color, lighting, scale, crop position, and image quality. Some images contained an isolated eye, nose, or mouth, while the face classes showed a wider portion of the animal's head. The nose and mouth classes sometimes included overlapping facial regions, making their visual boundaries less distinct. This later appeared in the model's confusion matrix, particularly between CatNose and CatMouth.



Figure 1. Distribution of training images across the eight target classes.

Data Quality Checks

Several checks were completed before preprocessing the images. All 63,660 training images and 29,443 test images could be opened successfully, and no corrupted files were found. The image identifiers in the label and submission files also matched the available image files, with no missing labels or duplicated identifiers.

The images varied considerably in size. There were 7,861 unique dimensions in the training set and 7,786 in the test set, showing that resizing was necessary before the images could be processed in batches. Common dimensions included 40 × 40 and 48 × 48 pixels, although many other sizes were present. Some images were also blurred, tightly cropped, or affected by differences in lighting and orientation.

A hash-based duplicate check identified 15 pairs of identical training images. The duplicate images within each pair had matching class labels, so there were no direct labelling conflicts. To prevent identical images from appearing in both the training and validation sets, one additional image from each pair was excluded before splitting the data. This left 63,645 unique training examples.

Representative samples from each class are shown in Figure 2. The samples illustrate the visual differences between full faces and individual facial features, as well as the variation within each class.



Figure 2. Representative training image from each target class.

Training and Validation Split

After removing the additional copies from the 15 duplicate pairs, the remaining 63,645 labelled images were divided into training and validation sets. A stratified 80/20 split was used, producing 50,916 training images and 12,729 validation images. Stratification preserved approximately the same class proportions in both sets, allowing the validation results to provide a more reliable estimate of performance across all eight classes.

A fixed random seed was used so that the same split could be reproduced whenever the notebook was rerun. Data augmentation was applied only to the training set, while the validation set used deterministic preprocessing. The separate Kaggle test set was not used during training, model selection, or hyperparameter decisions. This maintained a clear boundary between training, validation, and final test prediction, reducing the risk of data leakage.

Data Preprocessing and Pipeline Preparation

The original images came in many different sizes, so each image was converted to RGB and resized to 224 × 224 pixels. This size was chosen to match the input expected by EfficientNet-B0. The images were then converted to tensors and normalized using the standard ImageNet values, keeping the input consistent with the model's pretrained weights. The eight class names were also encoded as numerical labels from 0 to 7.

To help the model handle natural variation, the training pipeline included horizontal flipping, small rotations of up to 10 degrees, and mild color changes. These augmentations created slightly different versions of the training images while keeping the important facial feature recognizable. Random augmentation was not used for validation or test images, allowing them to be assessed consistently.

The images were loaded in batches of 32 using custom PyTorch dataset and data-loader objects (Paszke et al., 2019). Training images were shuffled between epochs, while the validation and test order remained fixed. Overall, this pipeline kept the processing consistent while adding enough variation to support better generalization.



Figure 3. Examples of training images after preprocessing and augmentation.

Model Selection and Architecture

EfficientNet-B0 was selected as the main classification model because it offers a practical balance between accuracy, model size, and training cost (Tan & Le, 2019). Its convolutional layers learn visual patterns at different levels, ranging from simple edges and textures to more detailed shapes. This made it suitable for distinguishing closely related facial features while still being manageable on the available hardware.

Transfer learning was used rather than training the model from scratch. The model began with weights learned from ImageNet, giving it a useful foundation for recognizing common visual features. Its original classification layer was replaced with a dropout layer set to 0.30 and a fully connected layer containing eight outputs, one for each target class. The model received 224 × 224 RGB images and produced raw class scores, with the highest score used as the predicted class.

This approach reduced the amount of task-specific training required and allowed the model to benefit from features learned from a much larger image dataset.

Architecture Details

EfficientNet-B0 builds its feature extractor from a sequence of mobile inverted bottleneck convolution blocks. These blocks progressively reduce the spatial size of the image while increasing the number of feature channels, allowing the network to move from simple patterns such as edges and color changes to more detailed shapes and textures. Squeeze-and-excitation components also help the network place greater emphasis on informative channels. Most internal blocks use the SiLU activation function.

The feature extractor produces 1,280 values for each image. In the adapted model, these values pass through a dropout layer with a rate of 0.30 before reaching the new eight-output linear layer. The output layer returns one score, or logit, for each class. Cross-entropy loss uses these logits directly during training, while the class with the highest score is selected during validation and test prediction.

Only 10,248 parameters were trainable during the first stage because the pretrained feature extractor was frozen. During fine-tuning, the final two feature blocks were unfrozen, increasing the number of trainable parameters to 1,139,640. This kept most of the pretrained representation stable while allowing the higher-level features to become more specific to cat and dog facial regions.

Training Strategy and Overfitting Management

Training Approach

Training was completed in two stages rather than adjusting the whole network at once. During the first three epochs, the pretrained feature-extraction layers were frozen, and only the new classification head was trained. This gave the new output layer time to adapt to the eight facial-feature classes without immediately changing the useful visual features already learned from ImageNet. AdamW was used with a learning rate of 0.001 and weight decay of 0.0001.

After this initial stage, the final two feature blocks were unfrozen and trained for another three epochs. A smaller learning rate was used during fine-tuning, 0.00001 for the feature layers and 0.0001 for the classifier. This allowed the model to adjust its higher-level features more carefully without making large changes to the pretrained weights.

The loss function was class-weighted cross-entropy with a label smoothing of 0.05. Class weighting gave more attention to the smaller classes, while label smoothing reduced the model's tendency to become overly confident. After every epoch, training and validation loss and accuracy were recorded. The checkpoint with the highest validation accuracy was saved as the final model.

Hyperparameter Justification

A batch size of 32 was chosen as a practical balance between stable gradient estimates, memory use, and training time on the available hardware. AdamW was selected because it adapts the learning rate for individual parameters while applying weight decay separately. The weight decay value of 0.0001 provided mild regularization without placing an excessive constraint on the model.

Class-weighted cross-entropy was used because the eye classes contained considerably more images than the smaller face, mouth, and nose classes. The weights were calculated from the training split, giving greater influence to errors from less frequent classes. Label smoothing of 0.05 was added to discourage extreme confidence and potentially improve performance on visually ambiguous images.

The learning rates were deliberately reduced during fine-tuning. The classifier could learn more quickly because it was newly initialized, while the pretrained feature layers required smaller updates to avoid damaging useful existing features. Training was limited to three epochs per stage, and the model with the highest validation accuracy

was saved rather than automatically using the final epoch. Together, these choices provided a controlled training process while keeping the computational demand manageable.

Training Behavior and Generalization

The learning curves showed steady improvement across all six epochs. Training accuracy increased from 74.40% to 88.77%, while validation accuracy rose from approximately 78% to 90.17%. At the same time, training loss fell from 0.9493 to 0.6161, and validation loss decreased from 0.8386 to 0.5846.

Validation performance remained slightly stronger than training performance. This is reasonable because the training images were made more challenging through random augmentation, and dropout was active during training but disabled during validation. Importantly, validation loss continued to fall rather than beginning to rise, so there was no clear sign of overfitting within the six completed epochs.

Fine-tuning produced the largest improvement. It allowed the later EfficientNet layers to adapt from general ImageNet features to the smaller details needed for recognizing animal eyes, mouths, noses and faces. The improving validation results suggest that this adjustment helped the model generalize rather than simply memories the training set.

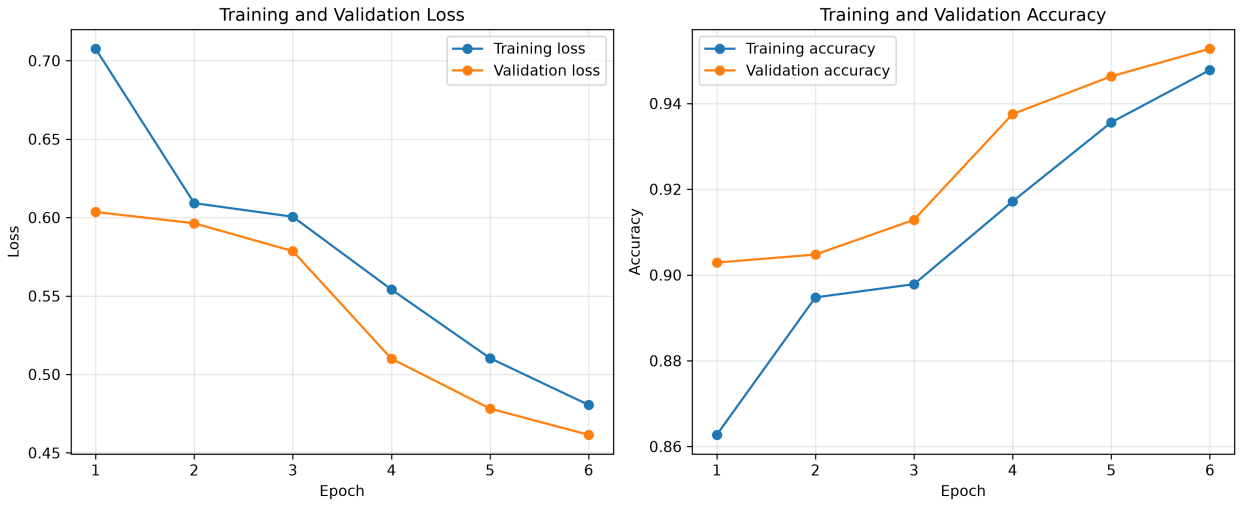


Figure 4. Training and validation loss and accuracy across six epochs.

Model Evaluation and Kaggle Submission

Validation Performance

The final checkpoint was evaluated using accuracy, balanced accuracy, and F1-score measures implemented with scikit-learn (Pedregosa et al., 2011). It achieved a validation accuracy of 90.17% and a balanced accuracy of 90.01%. The similarity between these values suggests that the model performed reasonably well across different class sizes rather than relying mainly on larger eye classes. The macro F1-score was 89.58%, while the weighted F1-score was 90.15%.

Performance was the strongest for the face classes. CatFace achieved an F1-score of 98.96%, followed by DogFace at 94.68%. CatNose was the most difficult class, with an F1-score of 81.57% and recall of 77.31%. This indicates that the model recognized complete faces very reliably but found some smaller, visually overlapping features more difficult to separate.

Metric	Result
Validation accuracy	90.17%
Balanced accuracy	90.01%
Macro F1-score	89.58%
Weighted F1-score	90.15%

Table 1. Overall performance of the final model on the validation set.

Confusion Matrix Analysis

The confusion matrix shows that most predictions fall along the main diagonal, meaning that the majority of validation images were classified correctly. The clearest errors occurred between mouth and nose classes. In particular, 235 CatNose images were predicted as CatMouth, while 80 CatMouth images were predicted as CatNose. For dogs, 96 DogNose images were classified as DogMouth, and 43 DogMouth images were classified as DogNose.

These mistakes are understandable because mouth and nose images can include overlapping facial regions, similar fur patterns, and tightly cropped features. In comparison, the face classes were rarely confused because they provided more complete visual information. The results suggest that the model learned the distinction

between cats and dogs well but occasionally struggled to separate nearby features from the same animal.

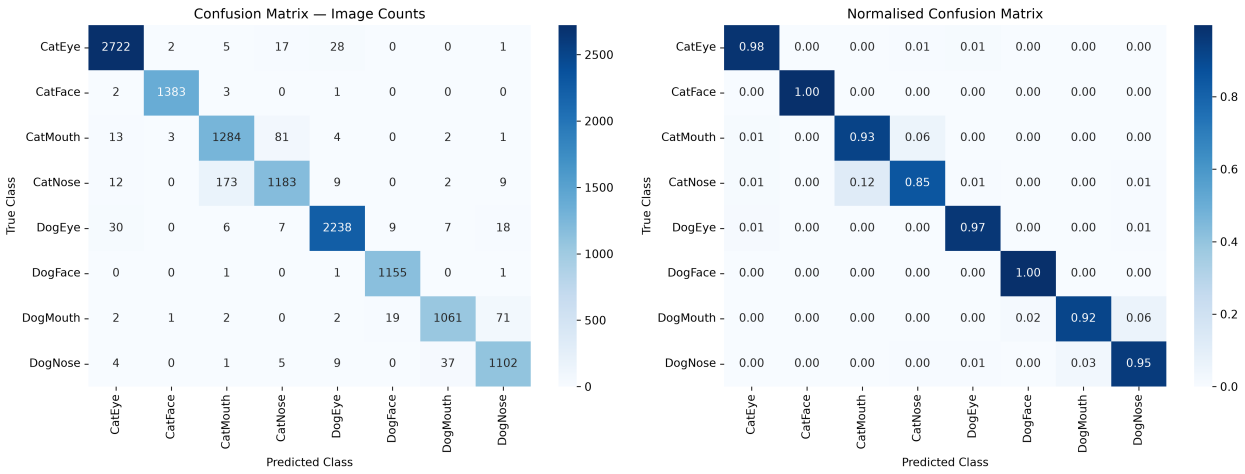


Figure 5. Validation confusion matrices showing class counts and normalized results.

Kaggle Submission Result

The final model was used to generate one prediction for each of the 29,443 test images. These predictions were added to a copy of the official sample submission file, preserving the required image order, column names, and class spelling. Before submission, the file was checked for missing values, invalid labels, and duplicate image identifiers.

Kaggle accepted the submission and returned a public leaderboard accuracy of 88.24%. This was 1.93 percentage points below the local validation accuracy of 90.17%. The difference is relatively small and suggests that the validation split gave a reasonable estimate of performance on unseen data. The lower Kaggle result may reflect differences in image difficulty or class distribution within the hidden test set.

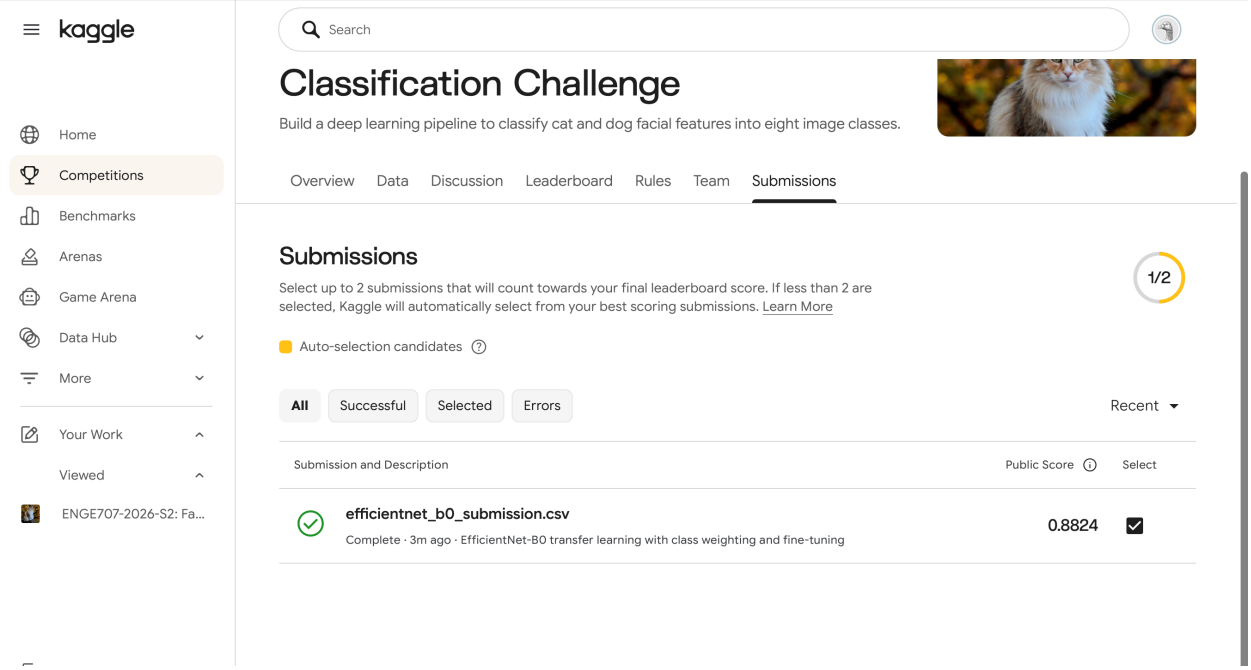


Figure 6. Successful Kaggle submission with a public leaderboard score of 0.8824.

Results Analysis and Improvement Strategies

Strengths and Limitations

The final pipeline was effective at learning the broad differences between species and facial regions. Transfer learning provided a strong starting point, while fine-tuning improved validation accuracy from 81.48% after the frozen-head stage to 90.17%. Data augmentation, dropout, and weight decay also supported generalization, with no clear divergence between training and validation performance.

The model's main weakness was its handling of closely related nose and mouth images. These classes sometimes contained similar crops or overlapping facial regions, giving the model less context than the complete face classes. Class weighting helped address the unequal class distribution, but it could not fully resolve this visual ambiguity.

A further limitation is that the model was evaluated using a single stratified split. Although the validation and Kaggle results were reasonably close, performance could vary with a different split. Training was also limited to six epochs and one main architecture. The results are somewhat encouraging, but they do not represent an exhaustive search for the best possible model.

Possible Improvements

The most useful next step would be to focus on the mouth and nose classes. More carefully selected augmentations could help the model cope with changes in crop position, lighting and sharpness without distorting the feature itself. Reviewing ambiguous training images could also reveal whether some crops contain too much of a neighboring facial region, although the official labels should not be manually changed without clear justification.

A longer fine-tuning schedule with early stopping and learning-rate scheduling could allow the model to improve while controlling overfitting. Hyperparameters such as dropout, label smoothing, batch size, and the number of unfrozen feature blocks could also be tested more systematically. Stratified cross-validation would provide a stronger estimate of generalization than a single split.

Finally, other pretrained architectures such as EfficientNet-B1, ConvNeXt or a vision transformer could be compared with EfficientNet-B0. Combining predictions from several well-performing models, or applying test-time augmentation, may improve Kaggle accuracy by reducing errors made by any one model.

Conclusion

This project developed a complete deep learning pipeline for classifying eight types of cat and dog facial features. Dataset exploration revealed differences in class size, image dimensions, and visual quality, which informed the preprocessing and training decisions. A consistent pipeline was created using image resizing, ImageNet normalisation, moderate augmentation and a stratified training–validation split.

EfficientNet-B0 provides a strong and practical foundation for transfer learning. Training the classifier first and then fine-tuning the later feature blocks improved validation accuracy from 81.48% to 90.17%. The final model achieved a balanced accuracy of 90.01% and a Kaggle public score of 88.24%, showing that it generalized reasonably well to unseen data.

The main errors occurred between mouth and nose classes, where cropped images often contained similar or overlapping information. Future improvements could focus on more systematic hyperparameter testing, longer fine-tuning with early stopping, cross-validation, and comparisons with other pretrained architectures. Overall, the final solution met the project objectives and produced a valid, reproducible Kaggle submission.

References

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00ahttps://doi.org/10.25080/Majora-92bf1922-00a

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*, 8024–8035. /bdbca288fee7f92f2bfa9f7012727740-Abstract.html/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning, 97*, 6105–6114. https://proceedings.mlr.press/v97/tan19a.htmlhttps://proceedings.mlr.press/v97/tan19a.html

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software, 6*(60), Article 3021. https://doi.org/10.21105/joss.03021https://doi.org/10.21105/joss.03021

Appendix

Appendix A – GitHub Repository

https://github.com/Asura-002/ENGE707-Facial-Feature-Classification.git

Appendix B – Kaggle Username

amanmohammed002 – Aman Mohammed